

4. Implementation

4.1. Codebase

The system is implemented in Python. The module `models.py` defines the typed node data classes and an embedding helper; `graph_manager.py` manages the Neo4j connection, the vector indexes, the typed create and link operations, and the audit primitives; `judge.py` implements the single-pass judge used both statelessly and with memory; `mcts_judge.py` and `mcts_retrieval.py` implement the two exploratory search components; and `mcts_pipeline.py` composes the evaluation modes. The driver `benchmark_leakage_free.py` runs the leakage-free benchmark, including the audit wrapper and the transient-error handling. The analysis scripts `analyze_stats.py`, `make_figures.py`, and `reproduce_all.py` regenerate every statistic and figure and verify every artifact.

4.2. The Memory-Assisted Judge

The Memory-Assisted Judge (MAJ) is a single-pass judge whose prompt is conditioned on retrieved memory. Retrieval proceeds in three stages, shown in Listing 4.1: contrastive past attempts, semantically similar issues, and recurring semantic patterns. Each stage queries the typed graph through a top-k similarity search; the retrieved material is then formatted into a single context block and supplied to the judge alongside the task and the candidate response.

```
def judge_with_memory(task, agent_output, graph_manager,
                      goal=None, model="gpt-4o"):
    code_embedding = get_embedding(agent_output)
    # Stage 1-2: contrastive attempts and similar issues
    contrastive = graph_manager.find_contrastive_attempts(
        code_embedding, top_k=3)
```

```

similar_issues = graph_manager.find_similar_issues(
    code_embedding, top_k=5)
# Stage 3: semantic patterns from the similar issues
semantic_patterns = graph_manager.find_semantic_patterns(
    code_embedding, top_k=3)
memory_context = _format_memory_context(
    contrastive, similar_issues, semantic_patterns)
# ... judge call conditioned on memory_context ...

```

Listing 4.1 – The three-stage memory retrieval of the MAJ judge.

4.3. The MCTS Components

The MCTS-Judge explores alternative reasoning trajectories over a tree of evaluation subtasks. At each step it selects the child node with the highest Upper Confidence Bound for Trees (UCT) score, which balances exploitation of high-reward branches against exploration of less-visited ones; the implementation is shown in Listing 4.2.

```

def uct_score(self, exploration_constant=3.0):
    """Upper Confidence Bound for Trees."""
    if self.visit_count == 0:
        return float('inf')
    parent_visits = self.parent.visit_count if self.parent else 1
    exploitation = self.q_value
    exploration = exploration_constant * math.sqrt(
        math.log(parent_visits) / self.visit_count)
    return exploitation + exploration

```

Listing 4.2 – The UCT selection score used by the MCTS-Judge.

The MCTS-Retrieval component replaces the fixed three-stage pipeline with a tree search over a set of retrieval actions, including four multi-hop graph traversals. The action set is defined declaratively, as shown in Listing 4.3, and each rollout assembles a trajectory of actions scored by the relevance, diversity, and volume of the material it retrieves. Both MCTS components are reported as exploratory; Chapter 5 shows they do not yield a statistically significant accuracy gain.

```

RETRIEVAL_ACTIONS = [
    {"name": "contrastive_attempts", ...},
    {"name": "similar_issues", ...},

```

```

    {"name": "semantic_patterns", ...},
    {"name": "multi_hop_issues_to_fixes", ...},
    {"name": "multi_hop_semantic_to_issues", ...},
    {"name": "multi_hop_policy_to_attempts", ...},
    {"name": "multi_hop_attempt_to_semantic", ...},
]

```

Listing 4.3 – The seven retrieval actions of MCTS-Retrieval, four of which are multi-hop graph traversals.

4.4. The Frozen-Memory Audit

The frozen-memory audit is the central engineering contribution of the implementation. Its first layer takes a deterministic snapshot of the memory graph before and after each evaluation. The snapshot records per-label node counts, per-relationship-type counts, and a SHA-256 fingerprint over the sorted node identifiers and edge triples, as shown in Listing 4.4. Two snapshots taken around a frozen-memory evaluation must be byte-identical; any divergence is direct evidence of a write that violates the leakage-free protocol.

```

def snapshot(self) -> dict:
    """Deterministic fingerprint of the current memory state."""
    import hashlib
    # ... collect node_counts, edge_counts, node_ids, edge_keys ...
    h = hashlib.sha256()
    h.update("|".join(node_ids).encode())
    h.update(b"\n")
    h.update("|".join(edge_keys).encode())
    return {
        "node_counts": node_counts,
        "edge_counts": edge_counts,
        "total_nodes": sum(node_counts.values()),
        "total_edges": sum(edge_counts.values()),
        "fingerprint": h.hexdigest(),
    }

```

Listing 4.4 – The memory snapshot and SHA-256 fingerprint.

The second layer prevents leakage rather than only detecting it. For the duration of an evaluation, every write method of the graph interface is intercepted by a context manager that raises a `FrozenMemoryViolation`

exception if any write is attempted. The audit is therefore defence-in-depth: the snapshot proves after the fact that nothing changed, and the write-blocker guarantees during the run that nothing can.

4.5. Structured Outputs

An early version of the MCTS-Judge extracted decisions with regular expressions over free-text model output, which failed silently on roughly 15 to 20 percent of responses that did not follow the expected template. All decision-extracting calls were migrated to a schema-validated structured-output interface; the schemas are shown in Listing 4.5. This eliminated parse failures without changing the semantics of any step.

```
class SubtaskDecision(BaseModel):
    analysis: str
    decision: bool          # True = correct, False = incorrect

class SelfAssessment(BaseModel):
    useful: bool           # True = subtask improves evaluation

class GlobalVerdict(BaseModel):
    verdict: bool          # True = response passes
    reasoning: str
```

Listing 4.5 – Pydantic schemas for structured judge outputs.

4.6. Handling Transient Failures

LLM evaluation is network-bound. In an early run, a mid-evaluation network outage caused a block of samples to error, and the original code recorded each errored sample as an incorrect answer, silently depressing accuracy. Two corrections were made. An exponential-backoff retry wrapper, shown in Listing 4.6, was added around every model call. And the error handling was changed so that a sample that still fails after retries is recorded with a null verdict and excluded from the accuracy computation, rather than counted as wrong. This correction is material: it is the difference

between an apparent catastrophic collapse under memory poisoning and the true, flat result reported in Chapter 5.

```
def _call_with_retries(fn, *, max_attempts=5, base=2.0,
                      max_sleep=30.0):
    """Retry fn() on transient errors with backoff + jitter."""
    for attempt in range(1, max_attempts + 1):
        try:
            return fn()
        except Exception as exc:
            if not _is_transient(exc) or attempt == max_attempts:
                raise
            sleep_s = min(max_sleep, base ** attempt) \
                + random.uniform(0, 0.5)
            time.sleep(sleep_s)
```

Listing 4.6 – Exponential-backoff retry wrapper for model calls.

4.7. Reproduction Pipeline

The benchmark driver builds memory for a chosen condition, runs the leakage-free evaluation with the frozen-memory audit enabled, and writes per-sample result files together with the audit records. The analysis script recomputes the Wilson intervals, the paired bootstrap intervals, and the exact McNemar tests from those per-sample files. The figure script regenerates every figure. A single reproduction script chains these steps, verifies that every expected result file and audit record is present, confirms that every audit passed, and exits with an error if any artifact is missing or any audit failed. Running that one script regenerates the entire empirical content of Chapter 5 from the raw per-sample data.